Tolerance-Aware Control Barrier Functions (TA-CBF)

A Complete Educational Guide — From First Principles to Hardware Deployment

Covering: Neural Networks · Point Clouds · SDFs · Neural ODEs · CLF · CBF · QP Control

---

Table of Contents

> **How to read this document.** Every concept is introduced from scratch, assuming only basic algebra. Worked numerical examples use concrete numbers. Formulas are given in math notation AND as plain text so you can follow both. Words shown like `this` are code or math symbols.

PART I — FOUNDATIONS

1.1 What is a Neural Network?

A **neural network** is a mathematical function that transforms numbers into other numbers. It is made of layers of simple operations stacked on top of each other. The network *learns* by adjusting internal numbers (called **weights** and **biases**) until its output matches what we want.

Think of it like a factory pipeline: raw ingredients go in, get processed at each station, and a finished product comes out. Each "station" is a **layer**.

The simplest possible network: a single neuron

A single neuron computes:

```
output = activation( w₁·x₁ + w₂·x₂ + ... + wₙ·xₙ + b )
```

where $\mathbf{x_1, x_2, ..., x_n}$ are the inputs, $\mathbf{w_1, w_2, ..., w_n}$ are the weights (what we learn), **b** is the bias (also learned), and **activation(·)** is a non-linear function explained below.

> **Worked Example — Single Neuron:**
> Inputs: $x_1 = 0.5$, $x_2 = -1.2$
> Weights (learned): $w_1 = 2.0$, $w_2 = -3.0$
> Bias: $b = 0.5$
>
> Step 1 — Linear combination (called the "pre-activation" or "z"):
> `z = 2.0 × 0.5 + (−3.0) × (−1.2) + 0.5 = 1.0 + 3.6 + 0.5 = 5.1`
>
> Step 2 — Apply activation (e.g. ReLU): `output = max(0, 5.1) = 5.1`

1.2 What is a Linear Layer (Fully Connected Layer)?

A **linear layer** (also called **fully connected** or **dense**) takes a vector of n numbers and produces a vector of m numbers by multiplying by a matrix of weights and adding a bias vector:

```
y = W · x + b

where  x ∈ ℝⁿ   (input vector,  n numbers)
       W ∈ ℝᵐˣⁿ (weight matrix, m×n numbers — all learned)
       b ∈ ℝᵐ   (bias vector,   m numbers — all learned)
       y ∈ ℝᵐ   (output vector, m numbers)
```

This is identical to doing the single-neuron calculation for m neurons simultaneously, all sharing the same input x but each with its own row of weights.

> **Worked Example — Linear Layer (2→3):**
> Input: `x = [1.0, 2.0]`
> Weight matrix W (3×2): `[[0.1, 0.4], [−0.2, 0.3], [0.5, −0.1]]`
> Bias b (3): `[0.0, 0.1, −0.2]`
>
> $y_1 = 0.1\times1 + 0.4\times2 + 0.0 = 0.1 + 0.8 = \mathbf{0.9}$
> $y_2 = -0.2\times1 + 0.3\times2 + 0.1 = -0.2 + 0.6 + 0.1 = \mathbf{0.5}$
> $y_3 = 0.5\times1 + (-0.1)\times2 + (-0.2) = 0.5 - 0.2 - 0.2 = \mathbf{0.1}$

> Output: `y = [0.9, 0.5, 0.1]`

1.3 Activation Functions: ReLU and Tanh

Without activation functions, stacking linear layers is pointless — the composition of linear functions is still linear. Activation functions add **non-linearity**, which lets the network represent curved, complex relationships.

ReLU — Rectified Linear Unit

```
ReLU(z) = max(0, z) = { z  if z > 0
                      { 0  if z ≤ 0
```

ReLU simply **kills negative values** (sets them to 0) and lets positives through unchanged. It looks like a hockey stick: flat on the left, diagonal on the right.

> ReLU(−5) = 0 ReLU(0) = 0 ReLU(2.3) = 2.3 ReLU(100) = 100

Why ReLU? Fast to compute. Gradients are either 0 or 1 — no "vanishing gradient" problem for positive inputs. Used in PointNet's per-point MLP because point features are non-negative similarities.

Tanh — Hyperbolic Tangent

```
tanh(z) = (eᶻ − e⁻ᶻ) / (eᶻ + e⁻ᶻ)

Output is ALWAYS between −1 and +1.
tanh(0) = 0,  tanh(−∞) = −1,  tanh(+∞) = +1
```

> tanh(−3) ≈ −0.995 tanh(−1) ≈ −0.762 tanh(0) = 0 tanh(1) ≈ 0.762 tanh(3) ≈ 0.995

Why Tanh? Outputs are bounded (−1 to +1), which prevents exploding activations. It is **smooth** everywhere (has a derivative at every point), which is essential when we need to differentiate through the network to compute barrier gradients. Used in ConditionalObstacleCBF and ResBlocks of the demo flow network.

Property	ReLU	Tanh
Output range	[0, ∞)	(−1, +1)
At zero	0	0
Derivative	0 if z<0, 1 if z>0	1 − tanh(z)² (always >0)

Smooth	No (kink at 0)	Yes
Bounded	No	Yes
Used in TA-CBF for	PointNet encoder	Barrier net, demo flow ResBlocks

1.4 How a Deep Network Transforms Information

A deep network is just many layers applied one after another. Each layer transforms the data into a new "representation" — a different set of numbers that hopefully captures something useful about the input. As you go deeper, features become more abstract.

```
Input x
  → Linear(2→128) → ReLU           [low-level features: "where is this point?
  → Linear(128→128) → ReLU         [mid-level: "is it near an edge?"]
  → Linear(128→64) → (no act.)     [embedding: compact 64-number summary]
Output: 64-dimensional feature vector (called an "embedding")
```

Information flow through a node

Every unit (node/neuron) in layer k:

1. **Receives** all activations from the previous layer
2. **Multiplies** each by its weight and sums (dot product)
3. **Adds** its bias
4. **Applies** the activation function
5. **Sends** its output to all nodes in the next layer

1.5 What is a ResNet Block (Residual Block)?

A ResNet block adds a **skip connection** that bypasses the internal transformation:

```
ResBlock(x):
    z   = tanh(W₂ · tanh(W₁ · x + b₁) + b₂)   ← standard MLP path
    out = z + x                                    ← add the original input ba
    return out
```

Why? The gradient of the loss can flow directly back through the skip connection (+1 gradient), preventing the "vanishing gradient" problem in deep networks. The block only needs to learn the *residual* (difference from the identity), which is often small and easy to learn.

> **Intuition:** If the ideal transformation is "barely change x", a plain MLP must learn to output approximately x from scratch. A ResBlock just needs to learn zero (the difference), which is much easier — and the skip carries the signal through unchanged.

1.6 How Networks Learn: Loss Functions and Backpropagation

The Loss Function

A **loss function** L(θ) measures how wrong the network is. A common choice for regression is **Mean Squared Error (MSE)**:

```
L = (1/N) Σᵢ ‖ŷᵢ − yᵢ‖²

ŷᵢ = network output for input i
yᵢ = correct (target) output for input i
N  = number of training examples
```

The loss is always a single non-negative number. Zero means perfect — the network predicts exactly the right answer every time. Our goal is to minimize it.

Backpropagation: How the Network Adjusts Weights

Backpropagation computes the **gradient** of the loss with respect to every weight in the network — that is, "if I increase this weight by a tiny amount, does the loss go up or down, and by how much?"

The algorithm uses the **chain rule** of calculus, working backwards from the loss through each layer:

```
∂L/∂W_layer_k = ∂L/∂output · ∂output/∂W_layer_k

(chain rule: gradient at layer k = gradient from above × local gradient)
```

Once gradients are computed, weights are updated by **gradient descent**:

```
W ← W − η · ∂L/∂W

η = learning rate (e.g. 0.001) — how big a step to take
```

> **Worked Example:** Suppose ∂L/∂w = +2.0 and η = 0.01
> → w decreases: w_new = w_old − 0.01 × 2.0 = w_old − 0.02
> → Moving w in the direction that DECREASES the loss.

1.7 What is a Point Cloud?

A **point cloud** is an unordered set of points in space, each described by its coordinates. In our 2D needle problem, the obstacle's shape is represented as a collection of (x, y) points sampled from its surface and interior.

```
Point cloud P = {p₁, p₂, ..., pₖ}  where each pᵢ ∈ ℝ²  (or ℝ³ in 3D)

Example for a blob obstacle:
  p₁ = (0.12, 0.08)
  p₂ = (0.15, 0.10)
  p₃ = (0.09, 0.11)
  ...  (K = 200 points total)
```

Challenge: A point cloud has no fixed order. Shuffling the points gives the same shape but a different input vector if we naively concatenate coordinates. Our network must be **permutation-invariant** — produce the same output regardless of point ordering.

Solution: PointNet architecture (Section 3.3) — process each point independently, then take the maximum across all points. The max-pool operation is permutation-invariant.

1.8 What is a Signed Distance Function (SDF)?

The **SDF** of a shape answers the question: "How far is point x from the shape's surface, and am I inside or outside?"

```
sdf(x) = { +d(x, surface)   if x is OUTSIDE the shape   (positive = safe)
         { 0                if x is ON the surface
         { −d(x, surface)   if x is INSIDE the shape   (negative = danger!)
```

> **Worked Example — Circular obstacle:**
> Center c = (0.2, 0.3), radius r = 0.05 m
>
> sdf(x) = ‖x − c‖ − r
>
> Point A = (0.27, 0.3): ‖A−c‖ = 0.07, sdf = 0.07 − 0.05 = **+0.02 m** (outside, safe)
> Point B = (0.2, 0.3): ‖B−c‖ = 0.00, sdf = 0.00 − 0.05 = **−0.05 m** (inside, danger!)
> Point C = (0.25, 0.3): ‖C−c‖ = 0.05, sdf = 0.05 − 0.05 = **0.00 m** (on surface)

The **gradient ∇sdf(x)** always points away from the shape surface (outward normal). Its magnitude is 1 almost everywhere (it's 1-Lipschitz). This is the key property that makes SDFs useful for safety: moving along +∇sdf moves you away from the obstacle.

1.9 Dynamical Systems and Differential Equations

A **dynamical system** describes how something evolves over time. In our case, the needle-tip position $x \in \mathbb{R}^2$ changes according to a velocity field:

```
ẋ(t) = f(x(t))    ← "the velocity at position x is f(x)"

ẋ = dx/dt = rate of change of position = velocity
```

Given the current position x(0) and this rule, we can compute where the needle will be at any future time by integration (numerically, using RK4 — see Section 4.2).

Equilibrium point: A point x* where f(x*) = 0. The system "stops" there. For trajectory tracking, the goal is to make the tracking error $e = x - x^*(t)$ converge to zero.

1.10 Lyapunov Stability: The V Function

A **Lyapunov function** V(x) proves that a dynamical system is stable (converges to the target) without solving the differential equation explicitly. Requirements:

```
V(x) > 0  for all x ≠ x*   (positive definite — measures "how far are we")
V(x*) = 0                      (zero at the goal)
V̇(x) < 0  along trajectories  (strictly decreasing — getting closer over time
```

Think of V as the "energy" of the system. If V always decreases, the system must eventually reach the goal (V=0). In our system:

```
V(e) = ‖e‖²  where  e = x − x*(s)  (tracking error, x* is current target on c

∇V = 2e   (gradient points away from zero error — toward current position)
V̇  = ∇V · ẋ = 2e · f(x)   (rate of change of V along the trajectory)
```

1.11 Control Barrier Functions: The B Function

A **Control Barrier Function (CBF)** B(x) guarantees that the system stays in a safe set S. If we define the safe set as $S = \{x : B(x) \geq 0\}$, then:

```
Safe set:  S = { x ∈ ℝ² : B(x) ≥ 0 }
           ∂S = { x : B(x) = 0 }  (boundary — the "fence")
           outside: B(x) > 0     (comfortably safe)
           inside:  B(x) < 0     (UNSAFE — obstacle penetration!)
```

For B to be a valid CBF, we require that any control u can maintain $B \geq 0$ by satisfying:

```
Ḃ(x) = ∇B(x) · (f(x) + u) ≥ −γ · B(x)

This says: the rate of decrease of B is bounded — B cannot drop "too fast."
If B is large (far from obstacle), we can afford Ḃ to be somewhat negative.
If B is small (near boundary), we need Ḃ ≥ 0 (can't decrease further).
```

Key insight: The CBF constraint says "we must be STEERING AWAY from the obstacle fast enough that we can't crash through." If B = 0.01 (very close), we need $\dot{B} \geq -\gamma \times 0.01$, which nearly forces $\dot{B} \geq 0$. If B = 1.0 (far away), the constraint is $\dot{B} \geq -\gamma \times 1.0$, which is very permissive.

PART II — PROBLEM SETUP

2.1 The Task: Needle Steering Through Foam

We have a Franka Research 3 (FR3) robot arm holding a thin needle. The needle must travel through a foam slab from a start point to a goal point, following a demonstration trajectory while **strictly avoiding 5 critical tissue obstacle shapes**:

- **star** — 5-pointed star with thin concave tips
- **crescent** — C-shaped with thin horn tips (hardest to avoid)
- **blob** — irregular convex blob
- **kidney** — concave kidney bean shape
- **lshape** — L-shaped right-angle obstacle

The state is the **2D needle-tip position** $x \in \mathbb{R}^2$ (in meters, measured in the foam plane). The control output is a **2D velocity** $\dot{x} \in \mathbb{R}^2$ (meters/second).

> **Why is this hard?**
>
> - Obstacle shapes are non-convex (star/crescent have thin concave regions)
> - Obstacles can appear at **arbitrary poses** (rotated, scaled, translated) — never seen during training
> - The needle must navigate around the obstacle while still reaching the goal
> - Dead-on approach causes "head-on deadlock" — no clear direction to go
> - Penetrating even 1mm into tissue can cause permanent damage

2.2 What We Need to Learn

Standard control theory can handle simple shapes analytically. But with non-convex, pose-varying shapes, we need three learned components:

Component	Symbol	What it computes	Why learned?
Demo Flow	$f_\theta(x, s)$	Nominal velocity toward demo trajectory	Complex curved demo paths from human demonstrations
Lyapunov Net	$V_\theta(e)$	Stability certificate $\|e\|^2$	Quadratic formula + small learned correction
Barrier Net	$B_\varphi(x)$	Safety certificate (distance-like to obstacle)	Non-convex shapes require learned representation

2.3 Inputs and Outputs Summary

```
At every control step (100 Hz = every 10ms):

INPUT:   x ∈ ℝ²   — current needle-tip position  (x, y in meters)
         s ∈ [0,1] — task progress (0=start, 1=goal)
         P ∈ ℝᴷˣ² — point cloud of obstacle shape (K=200 points)

COMPUTE: f_θ(x,s) ∈ ℝ²   — nominal velocity (where the demo flow wants to go)
         B_φ(x)   ∈ ℝ     — barrier value (how safe we are)
         ∇B_φ(x)  ∈ ℝ²   — barrier gradient (direction to move away from obst
         V(e)     ∈ ℝ     — Lyapunov value (how far from demo path)
         ∇V(e)    ∈ ℝ²   — Lyapunov gradient = 2·e

SOLVE:   QP → u ∈ ℝ²     — safety-filtered velocity correction

OUTPUT:  v_safe = f_θ(x,s) + u  — send to robot as velocity command
```

PART III — ARCHITECTURE IN DETAIL

3.1 Component A: ProgressConditionedDS — The Demo Flow Network f_θ

This network learns the **vector field** that drives the needle along the demonstration trajectory. It outputs a velocity $\dot{x} = f_\theta(x, s)$ pointing toward where the demo went next from position x at progress s.

What is "task progress" s?

$s \in [0, 1]$ is a scalar that encodes how far along the task we are. s=0 means we're at the start, s=0.5 means halfway, s=1 means at the goal. It is computed by finding the nearest point on the demonstration path and normalizing by path length.

Architecture Details

```
Input: [x₁, x₂, s]  ∈  ℝ³   (2D position + progress scalar)

Layer 1:  Linear(3 → 256)  +  Tanh
Layer 2:  ResBlock(256)     ← Linear(256→256)+Tanh+Linear(256→256) + skip cor
Layer 3:  ResBlock(256)
Layer 4:  ResBlock(256)
Layer 5:  Linear(256 → 2)  ← NO activation (velocity can be any real number)

Output: f_θ(x,s) ∈ ℝ²   (velocity: meters/second in x and y directions)
```

Layer	Input Dim	Output Dim	Parameters	What it learns
Linear + Tanh	3	256	3×256+256 = 1,024	Non-linear features of position+progress
ResBlock 1	256	256	256×256×2+512 = 131,584	Flow direction features
ResBlock 2	256	256	131,584	Curved path features
ResBlock 3	256	256	131,584	Goal-approach features
Linear (out)	256	2	256×2+2 = 514	Final velocity vector
Total	—	—	≈ 396,290	—

The Correction Term

In addition to the neural output, a linear correction term pulls the needle back to the demo path:

```
f_effective(x, s) = v_net(x, s) + K_f · (x_ref(s) − x)

K_f = DEMO_K = 5.0   (strong spring toward demo path)
x_ref(s)  = the reference position on the demo at progress s
x_ref(s) − x  = the error vector (how far we are from the demo path right now
```

This is like a **spring** attached to the demo path. Even if the neural network slightly drifts, the spring pulls the needle back. With K_f=5, a 10mm deviation creates a 0.05 m/s correction pull.

> **Worked Example — f_θ computation:**
> x = [0.35, 0.20] m (current needle position)
> s = 0.4 (40% along the task)
> x_ref(0.4) = [0.33, 0.18] m (demo position at 40% progress)
>
> Neural output: v_net = [0.02, 0.01] m/s (network says "go right and up a little")
> Correction: 5.0 × ([0.33,0.18] − [0.35,0.20]) = 5.0 × [−0.02, −0.02] = [−0.10, −0.10] m/s
>
> f_effective = [0.02−0.10, 0.01−0.10] = [−0.08, −0.09] m/s
> → Needle is pulled back toward the demo path (it drifted right and up).

3.2 Component B: LyapunovNet — V_θ(e)

The Lyapunov function measures how far the needle is from the demo trajectory. We use a **quadratic CLF** (Control Lyapunov Function) as in Nawaz et al. 2023:

```
V(e) = ‖e‖²  where  e = x − x_ref(s)   (tracking error)

V(e) = e₁² + e₂²   (sum of squared errors in x and y)

∇V(e) = 2e = [2e₁, 2e₂]   (gradient: closed-form formula, no neural net neede
```

There is also a small learned correction factor δ=0.05 times a correlation term, but for the purposes of the QP controller, the dominant terms are $V=\|e\|^2$ and $\nabla V=2e$.

> **Worked Example:**
> x = [0.35, 0.20], x_ref = [0.33, 0.18]
> e = [0.35−0.33, 0.20−0.18] = [0.02, 0.02]
> V = 0.02² + 0.02² = 0.0004 + 0.0004 = **0.0008** (very small → near demo path)
> ∇V = 2×[0.02, 0.02] = **[0.04, 0.04]**

3.3 Component C: CompositeBarrier — B_φ(x)

This is the most complex component. It takes the needle position and the obstacle's point cloud and outputs a scalar B ≥ 0 (safe) or B < 0 (inside obstacle). It has THREE sub-parts:

```
B_φ = CompositeBarrier = PART_A(encode) + PART_B(per-obstacle CBF) + PART_C(s

PART A: ObstacleEncoder   — maps point cloud → compact embedding vector
PART B: ConditionalObstacleCBF — maps (position + embedding) → scalar barrier
PART C: Smooth-min fusion — combines multiple bᵢ into one B(x)
```

Part A: ObstacleEncoder (PointNet-style)

This encodes the obstacle's shape from a point cloud into a fixed-size vector. It processes each point independently (so order doesn't matter), then takes the element-wise maximum.

```
Input: P = {p₁, ..., pₖ}  where each pᵢ ∈ ℝ²  (K=200 points, 2D coordinates)

For EACH point pᵢ independently (shared weights — same MLP applied to all):
  h₁ᵢ = ReLU( Linear(2 → 128) · pᵢ )          dim: 128
  h₂ᵢ = ReLU( Linear(128 → 128) · h₁ᵢ )       dim: 128
  h₃ᵢ = Linear(128 → 64) · h₂ᵢ                dim: 64  (no activation!)

Global aggregation (permutation-invariant):
  e = max_pool({h₃₁, h₃₂, ..., h₃ₖ})   ← element-wise max over all K points

  e ∈ ℝ⁶⁴   ← the "embedding" — 64 numbers summarizing the obstacle shape
```

The max-pool takes the maximum value at each of the 64 positions across all 200 points. Point i "wins" dimension j if it has the largest value there. This means: **whichever point most strongly activates each feature dimension gets to represent it**.

> **Why max-pool gives permutation invariance:**
> max(f(p_1), f(p_2), f(p_3)) = max(f(p_3), f(p_1), f(p_2)) [same result, any order]
> The embedding e does not depend on the order of points in the cloud. ✓

Layer (PointNet per point)	Input	Output	Activation	Parameters
point_mlp[0]: Linear	2	128	ReLU	2×128+128 = 384
point_mlp[1]: Linear	128	128	ReLU	128×128+128 = 16,512
point_mlp[2]: Linear	128	64	None	128×64+64 = 8,256

Max-pool over K=200 pts	K×64	64	—	0
Encoder total	K×2	64	—	≈ 25,152

Worked Example — PointNet step by step:

Say K=3 points (simplified): p_1=[0.1,0.2], p_2=[0.15,0.18], p_3=[0.08,0.22]

For p_1=[0.1, 0.2] (showing first 3 of 128 hidden units only):

$h_1^1 = \text{ReLU}([W_1 p_1 + b_1]) \rightarrow$ e.g. [0.23, 0.0, 0.51, ...] (128-dim)

$h_2^1 = \text{ReLU}([W_2 h_1^1 + b_2]) \rightarrow$ e.g. [0.18, 0.31, 0.0, ...] (128-dim)

$h_3^1 = [W_3 h_2^1 + b_3] \rightarrow$ e.g. [0.45, −0.12, 0.33, ...] (64-dim)

Similarly for $p_2 \rightarrow h_3^2$ and $p_3 \rightarrow h_3^3$ (different values, same W)

Max-pool over $\{h_3^1, h_3^2, h_3^3\}$:

e[0] = max(0.45, 0.52, 0.38) = **0.52** (p_2 wins dim 0)

e[1] = max(−0.12, −0.08, 0.10) = **0.10** (p_3 wins dim 1)

...

$e \in \mathbb{R}^{64}$ = overall obstacle embedding

Part B: ConditionalObstacleCBF — per-obstacle barrier $b_i(x)$

Given the embedding e_i from Part A and the relative position of the needle to the obstacle, this network outputs a single scalar $b_i(x)$ — the barrier value for obstacle i alone.

```
Input construction:
  x_rel = x − center_i          ∈ ℝ²    (relative position to obstacle i's cer
  x_norm = x_rel / σ            ∈ ℝ²    (normalized: σ=0.05m is typical obstac
  eᵢ                            ∈ ℝ⁶⁴   (obstacle embedding from Part A)

  concat → input ∈ ℝ⁶⁶         (stack: [x_norm₁, x_norm₂, e₁, e₂, ..., e₆₄])

Network layers (dims=[66+1=67] input, [256,256,256] hidden, [1] output):
  Linear(67 → 256) + Tanh       dim: 256
  Linear(256 → 256) + Tanh      dim: 256
  Linear(256 → 256) + Tanh      dim: 256
  Linear(256 → 1)               dim: 1   ← near-zero init (last layer weights

Output: bᵢ(x) ∈ ℝ¹    (scalar barrier for obstacle i)
  bᵢ > 0 → safe (outside obstacle i)
  bᵢ ≤ 0 → inside obstacle i (UNSAFE)
```

The **near-zero initialization** of the last layer is crucial: at the start of training, $b_i(x) \approx 0$ everywhere, which means the gradients that the loss pushes are well-defined from the very first step. A large random initialization would cause chaotic early training.

Layer (ConditionalObstacleCBF)	Input	Output	Activation	Parameters
Linear 1	67	256	Tanh	67×256+256=17,408
Linear 2	256	256	Tanh	256×256+256=65,792
Linear 3	256	256	Tanh	65,792
Linear 4 (output)	256	1	None	256×1+1=257
CBF net total	67	1	—	≈ 215,249

Part C: Smooth-min over Obstacles

If there are N obstacles, we have N barrier values $b_1(x)$, $b_2(x)$, ..., $b_n(x)$. The combined barrier B(x) must be safe with respect to ALL obstacles simultaneously. Naively, we'd take the minimum. But min() is not smooth (not differentiable where two b_i are equal), which breaks gradient computation. So we use the **smooth-min (log-sum-exp approximation)**:

```
B(x) = −(1/β) · log( Σᵢ exp(−β · bᵢ(x)) )    [smooth-min formula]

β = 1000   (smoothness parameter — higher β → closer to true min)

As β → ∞: smooth-min → exact min  (β=1000 is essentially exact for bᵢ values

Equivalent form:  B(x) ≈ min(b₁(x), b₂(x), ..., bₙ(x))  for large β
```

Why smooth-min instead of hard min?
The CBF constraint requires ∇B (the gradient of B). If $B = \min(b_i)$, the gradient is undefined wherever two b_i values are equal (non-differentiable point). The smooth-min is differentiable everywhere and converges to the true min as $\beta \to \infty$. With β=1000 and values in the centimeter range, the approximation error is below 10^{-43} — effectively zero.

Worked Example — Smooth-min with 3 obstacles:
b_1 = 0.015 (15mm from obstacle 1 — moderately safe)
b_2 = 0.003 (3mm from obstacle 2 — close!)
b_3 = 0.040 (40mm from obstacle 3 — far away)
β = 1000

> $\exp(-1000\times0.015) = e^{-15} \approx 3.06\times10^{-7}$
> $\exp(-1000\times0.003) = e^{-3} \approx 4.98\times10^{-2}$
> $\exp(-1000\times0.040) = e^{-40} \approx 4.25\times10^{-18}$
>
> Sum $\approx 4.98\times10^{-2}$ (dominated by obstacle 2 which has the smallest barrier)
> $B = -(1/1000)\cdot\log(4.98\times10^{-2}) \approx -(1/1000)\cdot(-3.0) \approx$ **0.003**
>
> → $B \approx \min(0.015, 0.003, 0.040) = 0.003$ ✓ (correct — obstacle 2 dominates)

Barrier Gradient $\nabla B_\varphi(x)$

The QP controller needs $\nabla B_\varphi(x)$ — the gradient of B with respect to x. This is computed using PyTorch's automatic differentiation (**autograd**):

```
Call model.B.gradient(x_tensor):
  1. x_tensor.requires_grad_(True)   ← tell PyTorch to track gradients
  2. B_val = model.B.forward(x_tensor)   ← forward pass
  3. B_val.backward()   ← backpropagation through ALL layers
  4. gradB = x_tensor.grad   ← ∂B/∂x₁ and ∂B/∂x₂

Result: gradB ∈ ℝ²   ← points in the direction of increasing B (away from obs
```

The gradient points in the direction that INCREASES B most rapidly — i.e., away from the obstacle. Moving in direction $+\nabla B$ means moving away from danger.

PART IV — TRAINING

4.1 Overview: Two-Stage Training Strategy

```
STAGE 1 (~2.5 hours on GPU):
  Train f_θ AND V_θ AND B_φ jointly
  · Pre-training: 300 epochs on imitation loss (f_θ learns demo paths)
  · Outer loop (10 iterations × 150 inner epochs):
      composite_barrier_loss (trains all three networks together)
  · counter-example (CEX) replay: hard examples where B failed stored in repl

STAGE 2 (~3-5 minutes on GPU):
  FREEZE f_θ and V_θ (stop updating their weights)
  Train ONLY B_φ for 12,000 steps using augmented_barrier_loss
  · Full pose/scale augmentation on point clouds and SDF labels
  · Purpose: teach B_φ to generalize to unseen obstacle poses/scales
```

4.2 Stage 1: Pre-training f_θ (Imitation Loss)

Before training the safety components, we train f_θ to reproduce the demonstrations. Given M demonstration trajectories, each with T time steps:

$$L_{imitation} = \frac{1}{M \cdot T} \sum_i \sum_k \|\hat{x}_i(t_k) - x_i(t_k)\|^2$$

$\hat{x}_i(t_k)$ = predicted position from integrating f_θ starting at $x_i(t_1)$
$x_i(t_k)$ = actual position from demonstration i at time t_k

What is RK4 (Runge-Kutta 4th order)?

To get $\hat{x}(t_{k+1})$ from $\hat{x}(t_k)$, we numerically integrate $\dot{x} = f_\theta(x, s)$:

```
Standard Euler (bad): x̂(t+dt) = x̂(t) + dt · f_θ(x̂(t), s)
  → Too inaccurate for small dt

RK4 (4th order, much better):
  k₁ = f_θ(x,          s)          ← slope at start
  k₂ = f_θ(x + dt/2·k₁, s+ds/2)   ← slope at midpoint using k₁
  k₃ = f_θ(x + dt/2·k₂, s+ds/2)   ← slope at midpoint using k₂
  k₄ = f_θ(x + dt·k₃,   s+ds)     ← slope at end using k₃

  x(t+dt) ≈ x(t) + (dt/6)·(k₁ + 2k₂ + 2k₃ + k₄)
```

RK4 evaluates f_θ FOUR times per step and combines them with a weighted average. The error is O(dt⁵) — much smaller than Euler's O(dt²).

4.3 Stage 1: Composite Barrier Loss

The barrier loss has three terms. First, let's define what target we're training B_φ to approximate — the **SDF-shaped target**:

```
Target for B_φ at position x near obstacle i:
  sdf_i(x)  =  signed distance from x to obstacle i's surface

  b_target(x) = K · clip(sdf_i(x) − Δ, −CLAMP_IN, +CLAMP_OUT)

  K         = BARRIER_SDF_K = 4.0      (amplification factor)
  Δ         = INFLATE_MARGIN = 0.010m  (10mm inflation — shift B=0 outward)
  CLAMP_OUT = 0.013 m                  (max positive target: 13mm * 4 = 0.052
  CLAMP_IN  = 0.040 m                  (max negative: goes down to −0.16)

Example: sdf = +0.005m (5mm outside surface)
  raw = 0.005 − 0.010 = −0.005   ← still "in the danger zone" (within inflati
  b_target = 4 × clip(−0.005, −0.040, +0.013) = 4 × (−0.005) = −0.020
```

This target design means B=0 is not at the obstacle surface — it's **10mm outside**. So B≥0 means "I'm more than 10mm outside" — strict avoidance with a built-in buffer.

```
TERM 1: L_safe  =  MSE of B_φ(x) vs b_target(x)  for SAFE points (sdf > INFLA
  "The network should output the correct positive value for points outside"

  L_safe = mean over safe samples: (B_φ(x) − b_target(x))²

TERM 2: L_unsafe = hinge loss for UNSAFE points (inside or in inflation shell
  "The network must predict NEGATIVE values here"

  L_unsafe = mean over unsafe samples: max(0, B_φ(x) + 0.001)²
  (zero if B_φ < −0.001, penalizes if B_φ ≥ −0.001)

TERM 3: L_dot = CLF decrease condition
  "The demo flow must satisfy Lyapunov decrease V̇ ≤ −α·V"

  L_dot = mean: max(0, ∇V·f_θ(x,s) + α·V(e))²  ← soft constraint

TOTAL Stage 1:  L = λ_FS·L_safe + λ_OB·L_unsafe + λ_DOT·L_dot
                + λ_reg·L_reg   ← L2 weight regularization of all networks
```

4.4 Why Freeze f and V in Stage 2?

> **Q: Why not train f, V, and B all together with augmentation in Stage 1?**
>
> **A: Stability and efficiency.**
>
> Stage 1 is a slow, careful co-training: f must learn the demonstration paths well, V must satisfy the Lyapunov conditions, and B must learn the basic obstacle geometry. These three objectives compete — gradients from the barrier loss can disrupt the demo-flow network's convergence.
>
> Stage 2 observes that once f and V are good (they don't change with pose augmentation — the demo path doesn't care about obstacle rotation), we can focus entirely on making B generalize. Freezing f and V means:
> 1. Faster training (fewer parameters to update — only B_φ's ~215k weights)
> 2. No risk of corrupting the already-good demo flow
> 3. We can do aggressive augmentation without worrying about f or V overfitting to a particular scene
>
> If we trained all at once, the optimizer would fight itself — B gradients would corrupt f, and f gradients would corrupt B's pose-generalization learning.

4.5 Stage 2: Augmented Barrier Loss

Stage 2 is the key to **zero-shot generalization**: the ability to avoid obstacle at poses never seen during training.

What is Pose Augmentation?

Before each training step, every obstacle in the scene is randomly transformed:

```
For each obstacle i, sample:
  θ  ~ Uniform(0, 2π)     ← random rotation angle
  σ  ~ Uniform(0.65, 1.4) ← random scale factor (65% to 140% of original size
  t  ~ Uniform(−0.05, 0.05) × 2  ← random translation (±5cm in x and y)

Rotation matrix:  R = [[cos θ, −sin θ], [sin θ, cos θ]]

Apply to point cloud:  p_augmented = σ · R · p_original + t
Apply to SDF label:   sdf(x) for augmented shape is computed analytically
                      (exact formula using R, σ, t applied in reverse)
```

The SAME transform is applied to the point cloud AND the SDF labels. This means: whatever shape the encoder sees in the point cloud, the target barrier values are computed from the correspondingly transformed

SDF. The network has no choice but to learn **invariant features** — representations that work for any rotation/scale.

Augmented Barrier Loss Formula

```
L_aug = λ_reg·(L_reg + L_comp)   ← regression + composite consistency
      + λ_FS·L_safe              ← correct positive values outside
      + λ_OB·L_unsafe            ← correct negative values inside
      + λ_DOT·L_dot              ← CLF decrease (Ḃ ≤ −α·V)
      + λ_EIK·L_eikonal          ← smooth gradient magnitude

where the EXTRA term (vs Stage 1):
  L_eikonal = λ_EIK · mean( ReLU(‖∇B_φ(x)‖ − B_GRAD_MAX)² )
  B_GRAD_MAX = 12.0

The eikonal term CAPS the gradient magnitude of B to B_GRAD_MAX=12.
```

Why the Eikonal (Gradient Cap) Term?

> **Critical Bug Without This Term:**
> Without the gradient cap, the hinge loss (L_unsafe) only forces B < 0 inside — it doesn't control HOW negative or HOW steep B is. The network discovered a shortcut: learn a near-STEP function: B≈+1 outside, B≈−1 inside, with an extremely thin transition (‖∇B‖ ≈ 290 m^{-1}!).
>
> With ‖∇B‖=290, the QP constraint ∇B·(f+u) ≥ −γ·B becomes very sensitive to tiny position errors. Near the obstacle, ∇B·f could be +290×0.01=+2.9 (large and positive from the demon flow pointing away) OR −290×0.01=−2.9 (large and negative if slightly misaligned). The QP would sometimes compute cbf_rhs < 0, meaning "no correction needed" — and the needle sailed through.
>
> The fix: cap ‖∇B‖ ≤ 12. Now the barrier looks like a proper SDF (gradual slope), and ‖∇B‖≈4 (=K) makes the QP well-conditioned.

4.6 Counter-Example (CEX) Replay Buffer

During training, the CEX buffer stores "hard" examples — positions where the barrier was wrong:

```
Add to CEX buffer when:  B_φ(x) > B_SAFE_MARGIN  BUT  actual sdf(x) < INFLATE
  (the network thinks x is safe, but x is actually inside the danger zone)

During each training step: 20% of the batch comes from the CEX buffer
  (the most important mistakes to fix — "tell the network about its failures"
```

PART V — THE ONLINE QP CONTROLLER

5.1 Overview: What Happens at Each Control Step

Every 10ms (100 Hz), the controller runs the following pipeline:

```
STEP 1: Read x (needle position from robot sensors)
STEP 2: Compute s (task progress) from x and demo path
STEP 3: Forward pass through f_θ(x, s) → nominal velocity f_val
STEP 4: Forward pass through B_φ(x) → barrier value B_num
         Autograd → barrier gradient gradB
STEP 5: Compute CLF terms: e = x − x_ref(s),  V = ‖e‖²,  gradV = 2e
STEP 6: [Optional] Go-around guidance g if B_num < B_ACTIVE=0.04
STEP 7: Solve QP (OSQP) → correction u_qp
STEP 8: u_safe = g + u_qp
STEP 9: project_safe(u_safe) → discrete-time safety projection
STEP 10: analytic_safety_filter → final geometric backstop
STEP 11: Send f_val + u_safe to robot
```

5.2 The QP Formulation

The core of the controller is a **Quadratic Program (QP)** — an optimization problem with a quadratic (squared) objective and linear constraints. We solve it ~100 times per second.

Decision Variables

```
z = [u₁, u₂, ε]  ∈ ℝ³

u = [u₁, u₂] ∈ ℝ²   — velocity correction (what we're solving for)
ε ≥ 0               — "slack variable" for the CLF constraint (explained belo
```

Objective: Minimize Effort + Slack

```
min_{u, ε≥0}   ‖u‖² + λ·ε²

= u₁² + u₂² + λ·ε²   (sum of squares of all three decision variables)

λ = LAMBDA_SLACK = 0.5   (penalty for using slack — don't relax CLF unless ne

Penalizing ‖u‖² means: make the SMALLEST possible correction to stay safe.
Penalizing ε² means: allow CLF relaxation only when truly necessary.
```

Constraint 1: CBF (HARD — must be satisfied exactly)

```
∇B_φ(x)ᵀ (f_eff + u) ≥ −γ · (B_φ(x) − m)

Expanding:  gradB · u  ≥  −γ·(B − m) − gradB·f_eff
           [this is the RHS, computed once before solving]

= cbf_rhs := −γ · (B_num − margin) − gradB @ f_eff

γ = GAMMA_CBF = 3.0
margin = B_SAFE_MARGIN = 0.008

If B_num = 0.020, γ=3, margin=0.008, gradB=[0.8, 0.6], f_eff=[0.02, 0.01]:
  cbf_rhs = −3.0×(0.020−0.008) − [0.8,0.6]·[0.02,0.01]
          = −3.0×0.012 − (0.016+0.006)
          = −0.036 − 0.022 = −0.058

Constraint: gradB·u ≥ −0.058
(i.e. [0.8,0.6]·[u₁,u₂] ≥ −0.058 — doesn't need much outward push since f_eff
```

Constraint 2: CLF (SOFT — can be relaxed by ε)

```
∇V(e)ᵀ (f_eff + u) ≤ −α · V(e) + ε

Rearranging:  gradV·u − ε  ≤  −α·V − gradV·f_eff
                              [= clf_rhs, computed once]

α = ALPHA_CLF = 4.0

If V = 0.0008, α=4, gradV=[0.04,0.04], f_eff=[0.02,0.01]:
  clf_rhs = −4.0×0.0008 − [0.04,0.04]·[0.02,0.01]
          = −0.0032 − (0.0008+0.0004)
          = −0.0032 − 0.0012 = −0.0044

Constraint: gradV·u − ε ≤ −0.0044
(if u is zero, gradV·0 − ε = −ε ≤ −0.0044 → ε ≥ 0.0044 — small slack needed)
```

Constraint 3: Slack Non-Negativity

```
ε ≥ 0   (slack can't be negative — it would "help" the CLF for free)
```

Why is the CLF "soft" (with slack)?

> **CLF soft vs CBF hard:**
>
> The CBF constraint CANNOT be relaxed — violating it means hitting an obstacle. The CLF constraint CAN be relaxed — violating it just means the needle doesn't converge to the demo path as fast as desired. Near an obstacle, the CBF constraint may force u to point AWAY from the demo path, which INCREASES V (moving away from the reference). Without slack, the QP would be infeasible (no u can simultaneously satisfy "move toward demo" AND "move away from obstacle"). The slack ε allows the CLF constraint to be violated by ε, resolving the conflict. The penalty $\lambda \cdot \varepsilon^2$ ensures we only use as much slack as necessary.

5.3 OSQP: How the QP is Solved

OSQP (Operator Splitting Quadratic Program) is a solver that takes the QP and finds the optimal $z = [u_1, u_2, \varepsilon]$ in milliseconds.

The standard QP form that OSQP solves:

```
min_{z}   (1/2) zᵀ P z + qᵀ z

subject to:   l ≤ A z ≤ u_bounds

In our case:
z ∈ ℝ³ = [u₁, u₂, ε]

P = diag(2, 2, 2λ) = [[2, 0, 0],      ← ‖u‖²+λε²: 2u₁²+2u₂²+2λε² (factor 2 ab
                      [0, 2, 0],
                      [0, 0, 2×0.5]]

q = [0, 0, 0]   ← no linear terms

A = [[gradB₁, gradB₂, 0    ],    ← CBF constraint row
     [gradV₁, gradV₂, −1   ],    ← CLF constraint row
     [0,      0,       1    ]]    ← ε ≥ 0 constraint row

l = [cbf_rhs, −∞, 0]             ← lower bounds
u = [+∞,  clf_rhs, +∞]           ← upper bounds
```

OSQP uses an ADMM (Alternating Direction Method of Multipliers) algorithm. With `warm_starting=True`, it reuses the previous solution as the starting point — this makes it converge in ~10-50 iterations instead of ~1000, keeping latency low.

Worked Example — Full QP Solution

Complete numerical example:

State: x = [0.28, 0.22], obstacle center = [0.30, 0.20], radius ≈ 0.03m

Computed quantities:

B_num = 0.018 (18mm effective clearance — close!)

gradB = [−0.72, 0.63] (pointing away from obstacle: left and up in this case)

f_val = [0.04, −0.01] (demo flow: go right and slightly down — TOWARD the obstacle!)

f_eff = f_val = [0.04, −0.01] (assume no go-around guidance)

V_num = 0.0006, gradV = [0.03, 0.02] (small tracking error)

CBF RHS:

cbf_rhs = −3.0×(0.018−0.008) − [−0.72,0.63]·[0.04,−0.01]

= −3.0×0.010 − (−0.0288 − 0.0063)

= −0.030 − (−0.0351) = −0.030 + 0.0351 = **+0.005**

→ Positive cbf_rhs means the demo flow OPPOSES the barrier! We NEED u to help.

CLF RHS:

clf_rhs = −4.0×0.0006 − [0.03,0.02]·[0.04,−0.01] = −0.0024 − (0.0012−0.0002) = **−0.0034**

QP solves for [u_1, u_2, ε] minimizing $\|u\|^2+0.5\varepsilon^2$:

CBF: $-0.72u_1 + 0.63u_2 \geq 0.005$ (must push outward)

CLF: $0.03u_1 + 0.02u_2 - \varepsilon \leq -0.0034$

$\varepsilon \geq 0$

Solution (approx): $u_1 = -0.004$, $u_2 = 0.003$, $\varepsilon = 0.001$

→ u pushes AWAY from obstacle (slightly left/up)

→ Final velocity: f_eff + u = [0.040−0.004, −0.010+0.003] = [0.036, −0.007]

→ Still mostly following the demo, but with a slight safety correction

5.4 Go-Around Guidance (Swirl)

A head-on approach to an obstacle creates a deadlock: the demo flow points directly at the obstacle, and the CBF pushes directly back. The net result is u ≈ 0 — the needle stalls.

```
When B_num < B_ACTIVE = 0.040 (approaching obstacle):
  Compute the outward normal: n = gradB / ‖gradB‖
  Compute a tangential direction: t = [−n₂, n₁, 0]  (90° rotation of n in XY)

  If dot(t, goal_direction) < 0:
      t = −t   (flip to go toward the goal, not away)
```

```
  closeness = max(0, (B_ACTIVE − B_num) / B_ACTIVE)  ← 1.0 when very close

  g = SWIRL_GAIN × ‖f_val‖ × closeness × t

  SWIRL_GAIN = 1.0

Then: f_eff = f_val + g   (the "guided" nominal velocity)
The QP uses f_eff instead of f_val.

Key property: gradB · g ≈ 0  (g is tangential → doesn't fight the barrier cor
```

5.5 project_safe: Discrete-Time Safety Projection

The CBF constraint bounds the **continuous-time derivative** $\dot{B}$. But we integrate with a finite step dt = 0.01s (Euler). It's possible that even with $\dot{B} \geq -\gamma B$, the actual next state x + dt·v has B < margin due to integration error.

```
project_safe(x, model, v_nom, dt):
  thresh = min(B(x), margin) − 1e-6

  if B(x + dt·v_nom) ≥ thresh:
      return v_nom   ← already safe, no modification needed

  Bisection (24 iterations): find largest α ∈ [0,1] s.t. B(x + α·dt·v_nom) ≥
    lo = 0,  hi = 1
    for i in range(24):
        mid = (lo+hi)/2
        if B(x + mid·dt·v_nom) ≥ thresh:  lo = mid
        else:                              hi = mid

  return v_nom × lo   ← scale down velocity, preserving direction (tangential
```

After 24 iterations: $|lo - hi| = 2^{-24} \approx 6 \times 10^{-8}$ — essentially exact.

5.6 Analytic Safety Filter: Geometric Backstop

Even after the learned CBF and project_safe, there can be residual errors (the learned B is approximate). The final layer is an exact geometric check using the **known** scene SDF:

```
analytic_safety_filter(x, v, dt, shapes, margin=SAFETY_SDF_MARGIN=0.011):

  Case 1: sdf_min(x + dt·v) ≥ 0.011   → accept v unchanged  (safe step)
```

```
  Case 2: sdf_min(x) < 0.011           → ESCAPE MODE
    (already inside the danger zone — learned CBF failed)
    Compute exact SDF gradient: g = ∇sdf(x) via finite differences
    Return v_escape = g/‖g‖ × min(VEL_CLIP, 0.05)  (escape outward at 5cm/s)

  Case 3: safe now but step would cross   → BISECTION (24 iterations)
    Find largest α s.t. sdf_min(x + α·dt·v) ≥ 0.011
    Return v × α   (scale down, preserve direction)
```

The margin 0.011m = INFLATE_MARGIN = 10mm ensures the needle stays outside the light-red zone.

PART VI — DEPLOYMENT AND RESULTS

6.1 Hardware Deployment on FR3

1. **Calibrate scene**: Photograph the foam slab, identify obstacle positions using computer vision or manual marking. Record SDF parameters (shape type, center, orientation, scale).
2. **Launch franka_ros2**: `ros2 launch franka_bringup franka.launch.py robot_ip:=192.168.1.17`
3. **Launch CRISP controller**: `ros2 launch crisp_controllers cartesian_impedance.launch.py`
4. **Load trained model**: `model = load_model("checkpoints/final_model.pt")`
5. **Set DEMO_K=5** (strong demo adherence) and `SWIRL_GAIN=1.0`
6. **Control loop (100 Hz)**: Read EE position → compute f_θ, B_φ, QP → send velocity
7. **Monitor B_num**: if B < B_SAFE_MARGIN, the analytic_safety_filter activates
8. **Stop condition**: ‖x − x_goal‖ < 0.005 m (within 5mm of goal)
9. **Emergency stop**: if B < −0.005 (inside obstacle), halt robot immediately

6.2 Performance Summary

Test	Obstacles	Penetrations	Reach Goal	Notes
Canonical demo	0	0	100%	Base case
Dead-center on-path	1	0	4/4	Hardest config — relies on swirl + analytic filter
Scattered random poses	6	0	3/5	2/5 pocket traps (obstacle=0, reach=False)
Dense generalization	4–8	0	varies	All outside light-red zone
Dynamic (slow motion)	4+1 mover	0	4/4	Predictive look-ahead 0.10s

6.3 Known Limitations

1. **Learned B is "compressed"**: ‖∇B‖ ≈ 1.5 vs target K=4. B=0 sits near the surface but B=margin lands ~25–40mm out. Dramatic deviation is expected (embraced for paper figures).

2. **Pocket traps**: The reactive QP controller is not a global planner. Dense obstacle configurations can trap the needle in a local minimum where every direction violates CBF or CLF simultaneously.
3. **Head-on deadlock residual**: SWIRL_GAIN=1 with noisy ∇B can cause slight stalling. Exact analytic filter resolves it but may create a brief pause.
4. **3D extension needed**: Current implementation is 2D (x,y in foam plane). Real needle insertion is 3D and requires extending the point cloud and SDF to $\mathbb{R}^3$.
5. **Obstacle counting**: The smooth-min composition is O(N) in number of obstacles. Dense scenes (>15 obstacles) may exceed 10ms QP solve time.
6. **Pose uncertainty**: If the calibrated obstacle pose has 3mm error, the 10mm margin covers it, but 10mm+ errors can cause near-misses.
7. **f/V quality dependency**: Stage 2 barrier training assumes a good f/V from Stage 1. If Stage 1 produces a poor demo flow, Stage 2 cannot compensate.

6.4 Complete System Summary

```
Given: x (position), s (progress), {Pᵢ} (point clouds), {SDFᵢ} (scene)

──────────────────────────────────────────────────────────────────

NEURAL NETWORK INFERENCE (every 10ms):

1. For each obstacle i:
   eᵢ = ObstacleEncoder(Pᵢ)                    ← PointNet: K×2 → 64
   bᵢ = ConditionalObstacleCBF(x_rel/σ, eᵢ)    ← MLP: 67 → 256×3 → 1

2. B = smooth_min({bᵢ})                        ← −(1/1000)·logsumexp(−1000·bᵢ
   gradB = autograd(B, x)                       ← 2D gradient via backprop

3. f_eff = f_θ(x, s) + K_f·(x_ref − x) + g     ← demo flow + spring + swirl

4. V = ‖x − x_ref(s)‖²,  gradV = 2·(x−x_ref)

──────────────────────────────────────────────────────────────────

QP CONTROLLER (OSQP, ~1ms solve):

min ‖u‖² + 0.5ε²
s.t. gradB·u ≥ −3·(B−0.008) − gradB·f_eff       [CBF, hard]
     gradV·u − ε ≤ −4·V − gradV·f_eff            [CLF, soft]
     ε ≥ 0

→ u* = optimal velocity correction

──────────────────────────────────────────────────────────────────

SAFETY LAYERS (sequential, each can only reduce velocity):
```

```
5. u_total = u* + g (go-around guidance)
6. v = project_safe(x, f_eff + u_total, dt)       ← bisection in learned B
7. v = analytic_safety_filter(x, v, dt, shapes)   ← bisection in exact SDF

_____________________________________________________________________

OUTPUT: send v to robot as Cartesian velocity command
```

> **Key Insight — Why This Works:**
>
> The three-layer safety hierarchy handles different failure modes:
> • The **CBF-QP** handles normal operation — continuous-time safety with graceful degradation
> • **project_safe** handles discretization error — finite step can violate continuous guarantee
> • **analytic_safety_filter** handles learned B errors — the exact geometric truth is always available
>
> No single layer is perfect, but the composition gives rigorous safety.

6.5 Parameter Reference

Parameter	Value	Meaning
INFLATE_MARGIN	0.010 m	B=0 sits 10mm outside true obstacle surface
B_SAFE_MARGIN	0.008	CBF defends B ≥ 0.008 (not B ≥ 0)
SAFETY_SDF_MARGIN	0.011 m	Analytic filter: keep sdf ≥ 11mm
GAMMA_CBF (γ)	3.0	CBF exponential decay rate
ALPHA_CLF (α)	4.0	CLF convergence rate
LAMBDA_SLACK (λ)	0.5	CLF slack penalty in QP objective
DEMO_K / K_f	5.0	Spring gain toward demo path
SWIRL_GAIN	1.0	Go-around guidance magnitude
B_ACTIVE	0.040	Swirl activates when B < 0.040
BARRIER_SDF_K	4.0	SDF amplification for barrier target
BARRIER_BETA (β)	1000	Smooth-min sharpness
EMB_DIM	64	PointNet embedding dimension

BARRIER_SDF_CLAMP_OUT	0.013 m	Max positive SDF target (outside)
BARRIER_SDF_CLAMP_IN	0.040 m	Max negative SDF target (inside)
B_GRAD_MAX	12.0	Eikonal cap on ‖∇B‖

6.6 Glossary

Term	Full Name	Meaning
CLF	Control Lyapunov Function	Proves stability / convergence to demo path
CBF	Control Barrier Function	Proves safety / obstacle avoidance
QP	Quadratic Program	Optimization with quadratic cost, linear constraints
SDF	Signed Distance Function	Distance to obstacle (+outside, −inside)
RK4	Runge-Kutta 4th order	Numerical ODE integrator (4 function evaluations per step)
ADMM	Alternating Direction Method of Multipliers	Algorithm OSQP uses to solve the QP
CEX	Counter-Example	Training sample where B was wrong (safety violation)
MSE	Mean Squared Error	Average of (prediction − truth)² over training samples
TA-CBF	Tolerance-Aware CBF	This project — CBF designed to tolerate pose variation
FR3	Franka Research 3	The 7-DOF robot arm used for deployment

— End of Document —

Generated from TA-CBF project at /home/stanny/franka_ros2_ws/src/Tolerance_Aware_CBF(TA-CBF)/